Static Detection of Role-Boundary Forgery in LLM Chat Templates

PromptABI contributors
Preprint

***Abstract*—A chat model receives a conversation as one token sequence. The chat template, a Jinja program distributed with the model, serializes the messages and inserts the markers that separate one role's turn from the next. When message content is interpolated without neutralization and the tokenizer maps a marker typed by a user to the same token id the template uses, the user can forge a turn: the model receives exactly the tokens of a genuine assistant or system turn. We examine this property using the template and tokenizer files, and we present PromptABI, a static checker that reports each attacker-reachable field, each marker it can reproduce, the template line, and the marker's token ids. PromptABI combines a bounded symbolic executor with exact witness paths, a may-taint analysis over the full Jinja AST, and a composition step that classifies each marker against the real tokenizer. On the 100 most-downloaded text-generation models on the Hugging Face Hub (51 distinct templates), an independent oracle that renders with `transformers` confirms token-exact forgery of an assistant turn for every model. PromptABI flags all of them in a median of 17 ms per template; of the markers it classifies as control tokens, 351 of 351 are reproduced in user content; 281 have matching genuine rendered occurrences and 70 have no reference occurrence in the probe conversations. We also evaluate a serving-side fix: tokenizing message content separately with `split_special_tokens` removes the forged control tokens in only 21 of 51 templates, because 26 templates use added tokens that are not marked special, while escaping all added tokens in content removes them in all 47 templates whose turn separator contains a control token.**

I. Introduction

Prompt-injection attacks place instructions in data that an LLM-integrated application passes to a model [1], [2], [3]. Most defenses assume that the application can tell the model which part of its input is trusted. Structured queries separate instructions from data with reserved delimiters [4]; preference optimization and instruction hierarchies train models to respect role privileges [5], [6]; spotlighting marks untrusted text [7]. Each relies on role and delimiter tokens that untrusted text cannot produce.

In open-weight deployments the component that produces those tokens is the chat template. A Hugging Face model repository ships a Jinja template in `tokenizer_config.json` or `chat_template.jinja`, and `transformers` [8], vLLM [9], and similar servers render the conversation through it and tokenize the resulting string. The template writes markers such as `<|im_start|>assistant` around each message's content. If the content is interpolated verbatim and the tokenizer matches the marker as an added token anywhere in the string, a user who types the marker produces the template's own token id, and the model sees a forged turn. This is injection in the sense of Su and Wassermann: untrusted input changes the parse structure of the generated program [10]. The problem has been raised in issue trackers [11] and used for jailbreaks [12], [13], and motivate checking structural exposure across deployed templates.

Our claim is that *role-boundary forgery is a property of the template and tokenizer files that static analyses can flag per field and marker, with rendering used to establish token-exact witnesses.* The contributions are:

- a definition of token-exact role-transition forgery and of per-marker forgery, separating control-token markers from plain-text markers (Section III);
- PromptABI, a checker that combines bounded symbolic execution, heuristic taint analysis on the Jinja AST, and tokenizer composition, and reports field, roles, template line, marker, and token ids (Sections IV and V);
- a study of the 100 most-downloaded Hub chat models, together accounting for 212 million downloads in 30 days, with an independent dynamic oracle, and an evaluation of two serving-side fixes (Section VI). Scripts, pinned model list, and raw outputs are released with the tool.

II. Background and Related Work

a) Chat templates: A chat template is a Jinja program evaluated in a sandboxed environment with `messages`, `tools`, special-token variables, and an `add_generation_prompt` flag. The Hugging Face environment enables `trim_blocks` and `lstrip_blocks`, adds `raise_exception`, and replaces `tojson` with `json.dumps(..., ensure_ascii=False)`, which leaves `<` and `>` unescaped. A fast tokenizer matches its added tokens in the rendered string before the subword model runs. The `split_special_tokens` option turns this off for tokens marked special, but added tokens can also be registered as non-special, and those are matched regardless. For plain text, subword tokenizers such as BPE [14] make a string's ids depend on its neighbors.

b) Prompt injection and defenses: Perez and Ribeiro introduced goal hijacking and prompt leaking [1]; Gre-

shake et al. showed indirect injection through retrieved content [2]; Liu et al. formalized attacks and benchmarked defenses [3]. StruQ separates prompt and data with reserved delimiters and a front end that keeps the delimiters out of the data [4]; SecAlign and the instruction hierarchy train role-aware behavior [5], [6]; spotlighting transforms untrusted text so the model can recognize it [7]. PromptABI checks the precondition these defenses share, that untrusted fields cannot reproduce role and delimiter tokens, for the templates that open-weight deployments use.

c) Special-token and template attacks: ChatBug shows that aligned models misbehave when inputs depart from the chat format the model was tuned on [12]. Virtual Context injects special tokens so that a model treats attacker text as its own output [13]. Both measure model behavior once the structure is forged. PromptABI reports candidate structural exposures before deployment, with tokenizer composition and rendering distinguishing their token-level behavior.

d) Template fuzzing: TemplateFuzz mutates system messages, role markers, delimiters, conversation history and generation hints, then uses model-response feedback to search for jailbreaks [15]. Its Sections 3.2–3.4 and 4.3 describe a behavioral attack objective; our diagnostic keeps the deployed template fixed and seeks field-to-marker candidates without model weights. This is a difference in threat model and output, not evidence that our diagnostic is more effective. The new direct-probe comparison below addresses a simpler weight-free alternative.

e) Static analysis of templates: Taint analysis is a standard way to find injection vulnerabilities in web applications, from Pixy for PHP [16] to the parse-structure definition of injection [10]. Samuel et al. infer context-sensitive sanitization for web templating languages [17]. PromptABI applies the source, sink, and sanitizer view to Jinja chat templates. The difference is that the meaning of a sink is decided by a tokenizer: whether a string is a role boundary depends on the ids the tokenizer gives it.

f) Structured generation: Grammar-constrained decoding libraries [18], [19], [20] constrain model output to a grammar over tokens. PromptABI also checks output-side artifacts, such as stop sequences and grammars, against the tokenizer; this paper evaluates the input-side role-boundary check.

III. Problem Definition

Let T be a template, $render_T(c)$ the string it produces for conversation c, and τ a tokenizer mapping strings to id sequences. A *field* f is a path into the conversation, such as `messages[*].content`; it is *attacker-reachable* when an untrusted party controls its value for some role (we use the user and tool roles). A *marker* is a string that T emits outside every field: a literal or a string constant that reaches an output, or a special token the template emits.

Role-transition forgery. Let g be the conversation [user: u, assistant: a] and s the substring of $render_T(g)$ between u and a. T and τ admit token-exact role-transition forgery when, for the single-message conversation $h = [\text{user: } u\, s\, a]$, the ids $\tau(render_T(h))$ agree with $\tau(render_T(g))$ on every token before the start of a. The model then receives an assistant turn that the user wrote.

Marker forgery. A marker m is forgeable through f when some value of f places m in the rendered string inside that value's span and τ gives this occurrence the same ids as an occurrence the template emits. If m is an added token with context-independent matching that the tokenizer does not split, τ maps it to one id in any context; we call m a *control-token marker*. Otherwise m is a *plain-text marker*, and equality depends on the surrounding characters.

IV. Approach

a) Marker catalog: PromptABI collects the strings a template can emit, namely literal text and string constants that reach an output. Constants used only in comparisons, filter arguments, or `raise_exception` messages are excluded. From these it extracts markers with a small set of patterns: angle-bracket and `<|...|>` tokens, bracketed tags such as `[INST]`, code fences, hash headers, plain-text role headers such as `Assistant:`, and short compound runs such as `<|im_start|>assistant`. Special tokens declared by the tokenizer that the template emits are added; declared tokens the template never emits, such as reserved tokens, are counted separately.

b) Bounded symbolic execution: For templates in a supported fragment (simple loops over `messages`, conditionals on role and index, `set`, and a fixed list of filters) PromptABI enumerates symbolic render paths up to two messages, one tool, and 128 paths, maps each path's segments to role regions, and reports a witness for each attacker-controlled expression and marker: the path, the rendered excerpt with the marker substituted, and the marker's offset. Outside the fragment or beyond the bounds the executor records an abstention and keeps the witnesses from the paths it explored.

c) Whole-template taint analysis: Production templates use `namespace()` objects, reversed and filtered loops, macros, string methods such as `split` and `strip`, and `raise_exception`. To cover the full language, PromptABI parses the template with Jinja2's own parser, without rendering, and computes a flow-insensitive may-taint over the AST, iterated to a fixed point. Sources are `messages`, `tools`, and `documents`; an attribute or item access on a message record yields a label such as `messages[*].content`. Taint flows through assignments (including namespace attributes and `set` blocks), loops, slicing, indexing, concatenation, formatting, conditional expressions, string methods, filters that return text, and macro calls, whose bodies are analyzed with the call's argument taints. Direct numeric or boolean outputs are excluded from sinks; their taint is retained for downstream

transformations. Selection filters preserve taint. Replacement preserves candidate taint: deleting a marker can join its surrounding characters into another occurrence. Jinja's HTML-escaping filters neutralize directly interpolated string markers that contain `<`, `>`, `&`, or quotes. `tojson` is not a sanitizer under the Hugging Face environment. Each sink records the role guards that dominate it (`message.role == 'assistant'` and its negations), so a finding names the roles whose content reaches the sink, and sinks reachable only from assistant or system content are reported separately.

d) Tokenizer composition: For each finding PromptABI loads `tokenizer.json` with the `tokenizers` library, encodes the marker, and checks whether it is a single added token with `single_word=False` and no longer overlapping added token, and whether `tokenizer_config.json` sets `split_special_tokens`. The marker is classified as *control-token*, *split-by-config*, or *text*. This step reads the tokenizer but never renders the template.

V. Implementation

PromptABI is a Python package with no required dependencies; the taint analysis uses `jinja2` and the composition step uses `tokenizers`. The symbolic executor and role-region model are about 2,400 lines, the taint analysis about 580, and the tokenizer composition about 130. `promptabi template-scan DIR` reads a model directory with the file precedence of `transformers` (`chat_template.jinja`, then `chat_template.json`, then `tokenizer_config.json`) and prints one line per finding, for example

```
messages[*].content (roles user, line 23) can render
       '<tool_call>' [control-token [151657]]
```

It exits non-zero when user content can render a control-token marker, so it can gate a CI job. The symbolic check is also the `role-boundary-nonforgeability` rule of `promptabi verify`, which emits SARIF.

VI. Evaluation

RQ1 (applicability and cost): on how many deployed templates does each analysis produce a verdict, and at what cost? **RQ2** (model-level detection): does the static verdict agree with token-exact role-transition forgery observed by rendering? **RQ3** (marker-level precision): how many reported markers can user content reproduce at the token level, and does tokenizer composition predict which? **RQ4** (fixes): does tokenizing message content separately remove the forgery without changing benign inputs?

A. Setup

a) Subjects: The Hub analysis uses checker revision `4131479`; the synthetic boundary controls evaluate the revised implementation. On 2026-10-07 we listed Hub models with pipeline tag `text-generation` by 30-day downloads and kept, in rank order, each repository that ships a non-empty chat template and a `tokenizer.json` at its current commit, until we had 100; this reached rank 157. Of the 57 repositories skipped, 23 were GGUF-only, 17 had no chat template (mostly base models), 7 had no `tokenizer.json`, and 10 were gated without access for our account. The 100 models use 51 distinct templates, because quantized and fine-tuned repositories usually copy their base model's template; we report per-template counts where duplication would inflate them. Every file is pinned by commit sha and sha256.

b) Dynamic oracle: The oracle shares no code with PromptABI. It loads each tokenizer with `transformers` 5.19.0 (falling back to `PreTrainedTokenizerFast` when a repository requires remote code), renders with `apply_chat_template`, and tokenizes with default settings. For RQ2 it computes the separator s of Section III with $u =$ `Hello there` and a unique string a, adding a system message when a template rejects the first form, and tests token-exact agreement. For RQ3 it injects each reported marker m into a user message as `a` m `b` and compares the ids that cover m with the ids at each place the template emits m in three genuine conversations: plain, with a system message, and with a tool call and tool result. If the ids differ it retries with the two characters that surround a genuine occurrence, as an attacker would. The outcome is *confirmed*, *confirmed with context*, *confirmed special* (the marker is one of the tokenizer's added tokens but the probe conversations do not emit it, as with vision or tool-response tokens), *not confirmed*, or *unobserved* (a plain-text marker the probe conversations never emit, so there is no reference occurrence).

B. RQ1: Applicability and cost

The taint analysis parsed all 51 templates with Jinja2's parser and took a median of 17 ms per template, 2.5 s for the largest; classifying a marker against a loaded tokenizer took about 1.00 ms. The symbolic executor produced witnesses for 31 models (11 templates), 7 of them fully explored within bounds, in a median of 0.01 s. On the other 40 templates its fragment does not cover the top-level control flow, typically a `namespace()` declaration or a reversed loop over `messages`, and it abstains. The taint analysis is therefore the analysis that scales to the Hub, and the symbolic executor supplies exact render-path witnesses where its fragment applies. None of the 100 tokenizer configurations sets `split_special_tokens`.

C. RQ2: Role-transition forgery

The oracle found token-exact role-transition forgery in all 100 models and all 51 templates. In 47 templates the forged separator contains at least one added token, for example `<|im_end|>\n<|im_start|>assistant\n` in the ChatML family, `<|end|><|start|>assistant <|channel|>final<|message|>` in gpt-oss, and `<|Assistant|>` (with full-width bars) in DeepSeek-V3; in the other 4 it is plain text, such as `[/INST]` in

TABLE I
STATIC VERDICT AGAINST THE ORACLE'S TOKEN-EXACT ROLE-TRANSITION FORGERY. EVERY SUBJECT IS FORGEABLE, SO THE TABLE MEASURES RECALL.

	flagged	not flagged	recall
Taint, models	100	0	1.00
Taint, templates	51	0	1.00
Symbolic, models	31	69	0.31
Symbolic, templates	11	40	0.22

TABLE II
ORACLE OUTCOME FOR THE TAINT ANALYSIS'S MARKERS, PER DISTINCT TEMPLATE, BY TOKENIZER-COMPOSITION CLASS. GENUINE-REFERENCE MATCHES AND ADDED-TOKEN REACHABILITY WITHOUT A REFERENCE ARE REPORTED SEPARATELY.

	control-token	text
confirmed	280	158
confirmed with context	1	209
reachability only	70	4
not confirmed	0	27
unobserved	0	65

a Mistral tokenizer where the instruction tags are not added tokens, or `Assistant:` in DeepSeek-Coder-V2.

Table I shows that the taint analysis flags every model and the symbolic executor flags the 31 models whose templates its fragment covers. Because the sample contains no template that resists the forgery, model-level precision is not measurable here; RQ3 measures precision per reported marker instead.

D. RQ3: Marker-level outcomes

The oracle checked every marker either analysis reported for `messages[*].content`, per distinct template. Of the symbolic executor's markers, 741 of 744 decided markers were confirmed (0.996); of the taint analysis's, 722 of 749 (0.964). Table II splits the taint analysis's markers by tokenizer class.

For control-token markers, 281 findings matched a genuine rendered occurrence (280 directly and one with context), and 70 additional findings reproduced an added-token id in user content without a genuine reference occurrence. These 70 outcomes establish token reachability rather than a rendered role-boundary comparison. For text-classified markers, 367 of 394 findings with genuine reference occurrences matched (209 after reproducing neighboring characters); four additional outcomes established added-token reachability without a reference, and 65 were unobserved. These are selected finding-level agreements, not production specificity estimates. The 27 unconfirmed text markers are mostly tags inside tool instructions, such as `</IMPORTANT>`, whose boundary BPE pieces merge with the neighboring characters differently in user content than at the template's own occurrence, where the next character belongs to a control token. Tokenizer composition thus gives a severity split: a CI gate on control-token findings raised no unconfirmed alarm in this sample, and text findings form a short review list.

E. RQ4: Serving-side fixes

Both fixes render the conversation with a unique placeholder per message, tokenize the template's own text normally, and tokenize each message's content separately. The first tokenizes content with `split_special_tokens=True`. The second also encodes any added-token string found in content one character at a time.

For benign conversations both fixes reproduce ordinary tokenization exactly in 47 of 51 templates; in the other four, tokenizing content separately changes a token at a segment boundary. With `split_special_tokens` alone, every control token of the forged separator is removed from the user's span in 21 of 51 templates. In 26 templates some separator tokens survive because they are registered as non-special added tokens (for example `<think>` in the Qwen3 family), and in 6 of them the role tokens themselves are non-special, so the forged turn remains token-exact; these are the DeepSeek-V3 and R1 templates and one derivative, whose `<|User|>` and `<|Assistant|>` tokens are added tokens with `special=false`. Escaping all added tokens removes every separator control token in all 47 templates whose separator contains one, leaving no token-exact forgery among them. The 4 templates with plaintext separators need a template-side change, because no tokenizer setting distinguishes a typed `Assistant:` from the template's.

F. Frozen synthetic direct-probe comparison

We froze 20 synthetic Jinja constructions and six payloads before execution (`research/wave2/plans/probe.md`). A synthetic tokenizer registers `<|turn|>` as a context-independent added token; an exposure is an extra occurrence of that token compared with rendering empty content. The six-payload portfolio includes a complete marker, two fragments, a numeric value, a replacement-collapse input and benign text. Across 140 renders, 12 constructions have witnessed exposure and eight have no witnessed exposure under this portfolio. A single complete-marker probe witnesses 10 of the 12; it misses HTML-fragment and numeric reconstruction. The original static implementation flags 10 exposed cases, misses two boolean-selection cases, and flags two unwitnessed cases (full angle-bracket replacement and uppercasing). Jinja's `and`/`or` return selected operands rather than booleans; preserving operand taint repairs both misses. The repaired analysis flags all 12 exposed cases and retains two conservative warnings. This rerun is regression evidence after repair, not a held-out accuracy result. The six-payload baseline witnesses all 12 by definition of the observation label; its zero misses therefore is not independent accuracy evidence. It is a competent low-cost alternative, not an oracle for arbitrary input.

Median observed times are 0.63 ms for static analysis and 1.66 ms for portfolio rendering plus tokenizer checks on this machine, with compilation included. These single-pass measurements support no hardware-independent speed claim. The suite reveals a complementary workflow: use static diagnostics for field attribution and rendering portfolios for witnesses; neither a negative warning nor finite unsuccessful probes certify a sanitizer. An independent post-freeze review additionally found comparison and negation values converted back to strings that reconstruct the marker; dependency propagation repairs those counterexamples. These are post-result regressions, not added held-out observations. This synthetic evidence establishes no production precision or downstream safety improvement.

G. Threats to validity

The sample is one snapshot of one ranking and omits gated models we could not access and GGUF-only repositories, whose templates are embedded in the weight files. Per-model counts are dominated by a few template families, which is why we also report distinct templates. The oracle renders fixed conversation shapes: an *unobserved* marker may occur in a conversation we did not render, and *not confirmed* covers the two contexts we tried. The marker patterns, in particular the plain-text role-header pattern, were refined while inspecting this sample, so the recall figures are not a held-out estimate. The taint analysis is heuristic: parsing the full Jinja AST does not establish soundness for arbitrary transformations. A fixed synthetic stress test (artifact `study/audit_stress.py`) checks eight sanitizer cases: five single-pass replacement cases reconstruct a removed marker, so replacement is treated conservatively. Two added-token settings with `single_word=True` also fail context-independent matching and are classified as text. These diagnostics are synthetic boundary checks. The property is structural; we measure that the model receives a forged turn, and prior work measures how models respond to one [12], [13].

VII. Discussion

None of the templates in the sample neutralizes message content, so the protection of role boundaries in these deployments rests entirely on the serving layer. `apply_chat_template` with default settings passes the forgery through. Our structural results motivate two changes, whose downstream safety effects remain unmeasured. Servers that accept untrusted message content should tokenize it separately from the template's text and escape every added token, special or not; RQ4 shows that `split_special_tokens` alone is insufficient for models that register role tokens as non-special. Model publishers that use plain-text role headers should either move to added tokens or escape the header strings in the template. PromptABI's `template-scan` reports both cases and distinguishes them through the tokenizer class of each finding.

VIII. Conclusion

Template and tokenizer analysis identifies candidate role-boundary forgery, which rendering can confirm. PromptABI analyzes all 51 distinct templates of the 100 most-downloaded Hub chat models in milliseconds per template, flags every model that the rendering oracle shows to be forgeable, and all 281 control-token findings with genuine rendered reference occurrences match at the token level. The same study shows that the common serving-side switch, `split_special_tokens`, does not cover role tokens registered as non-special, and that escaping all added tokens in content does.

References

[1] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[3] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium*, 2024, arXiv:2310.12815.

[4] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium*, 2025, arXiv:2402.06363.

[5] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," *arXiv preprint arXiv:2410.05451*, 2024.

[6] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[7] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.

[8] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[10] Z. Su and G. Wassermann, "The essence of command injection attacks in web applications," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2006, pp. 372–382.

[11] huggingface/transformers contributors, "Avoiding prompt injection using special tokens via `apply_chat_template`," GitHub issue #29279, https://github.com/huggingface/transformers/issues/29279, 2024, opened 2024-02-25; open as of 2026-10-06.

[12] F. Jiang, Z. Xu, L. Niu, B. Y. Lin, and R. Poovendran, "ChatBug: A common vulnerability of aligned LLMs induced by chat templates," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2406.12935.

[13] Y. Zhou, L. Lu, H. Sun, P. Zhou, and L. Sun, "Virtual context: Enhancing jailbreak attacks with special token injection," *arXiv preprint arXiv:2406.19845*, 2024.

[14] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725.

[15] Q. Shen, Z. Xiao, L. Huang, E. Hu, Y. Tian, and J. Chen, "Templatefuzz: Fine-grained chat template fuzzing for jailbreaking and red teaming llms," 2026. [Online]. Available: https://arxiv.org/html/2604.12232v1

[16] N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A static analysis tool for detecting web application vulnerabilities," in *2006 IEEE Symposium on Security and Privacy (S&P)*, 2006, pp. 258–263.

[17] M. Samuel, P. Saxena, and D. Song, "Context-sensitive autosanitization in web templating languages using type qualifiers," in *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)*, 2011, pp. 587–600.

[18] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[19] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen, "XGrammar: Flexible and efficient structured generation engine for large language models," *arXiv preprint arXiv:2411.15100*, 2024.

[20] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without fine-tuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 10 932–10 952.